Métodos de Análisis de Datos 1

Unidad 1: Obtención y Preparación de Datos

Clases 3 a 7

Objetivos de la unidad

Al finalizar esta unidad, los estudiantes serán capaces de:

- Identificar y acceder a diferentes fuentes de datos
- Cargar datos desde archivos, bases de datos y APIs
- Evaluar la calidad de los datos
- Detectar y tratar valores faltantes, duplicados e inconsistencias
- Transformar datos para análisis
- Normalizar y estandarizar variables
- Integrar múltiples fuentes de datos
- Obtener datos mediante web scraping
- Organizar datos en formato tidy

1 Clase 3: Fuentes de Datos y Carga

1.1 Fuentes de datos

Los datos pueden provenir de múltiples fuentes. Es fundamental saber cómo acceder a cada una de ellas.

1.1.1 Archivos planos

Los formatos más comunes son:

- **CSV (Comma-Separated Values):** Valores separados por comas
- **TSV (Tab-Separated Values):** Valores separados por tabulaciones
- **Excel (XLSX, XLS):** Hojas de cálculo de Microsoft Excel
- **JSON (JavaScript Object Notation):** Formato estructurado común en APIs web
- **XML (eXtensible Markup Language):** Datos jerárquicos estructurados

Ventajas y desventajas de CSV

Ventajas:

- Simple y ligero
- Compatible con casi cualquier herramienta
- Fácil de inspeccionar en un editor de texto

Desventajas:

- No preserva tipos de datos (todo es texto)
- Problemas con delimitadores en el contenido
- No soporta múltiples hojas/tablas
- Sin compresión nativa

1.1.2 Bases de datos

Las bases de datos relacionales almacenan datos en tablas estructuradas:

- **SQLite:** Base de datos local, sin servidor, ideal para proyectos pequeños
- **PostgreSQL:** Base de datos robusta y de código abierto
- **MySQL/MariaDB:** Muy populares en aplicaciones web
- **SQL Server:** Solución empresarial de Microsoft

Para acceder a bases de datos, se utiliza **SQL (Structured Query Language)**.

1.1.3 APIs (Application Programming Interfaces)

Las APIs permiten acceder a datos de servicios web de forma programática:

- **REST APIs:** Usan HTTP (GET, POST, PUT, DELETE)
- **GraphQL:** Permite consultas más flexibles
- **Autenticación:** Muchas APIs requieren API keys o tokens

Ejemplo de API pública

OpenWeatherMap: API para obtener datos meteorológicos
URL de ejemplo:

```
https://api.openweathermap.org/data/2.5/weather?
    q=BahiaBlanca&appid=TU_API_KEY
```

Retorna un JSON con temperatura, humedad, presión, etc.

1.1.4 Web scraping

Cuando los datos están en páginas web sin API disponible, se puede usar **web scraping**:

- Descarga el HTML de la página
- Extrae la información usando selectores CSS o XPath
- Transforma los datos en formato estructurado

Advertencia sobre web scraping

Consideraciones legales y éticas:

- Revisar los términos de servicio del sitio
- Respetar el archivo robots.txt
- No sobrecargar el servidor (rate limiting)
- Verificar si existe una API oficial

1.2 Carga de datos en Python

1.2.1 Pandas para archivos

La librería `pandas` es la herramienta principal para cargar y manipular datos en Python.

CSV:

```
import pandas as pd

# Cargar CSV
df = pd.read_csv('datos.csv')

# Con opciones
df = pd.read_csv(
    'datos.csv',
    sep=',',                # Delimitador
    encoding='utf-8',       # Codificaci n
    na_values=['NA', ''],   # Valores que representan NaN
    parse_dates=['fecha']   # Parsear columnas como fechas
)
```

Excel:

```
# Cargar Excel
df = pd.read_excel('datos.xlsx', sheet_name='Hoja1')

# M ltiples hojas
todas_las_hojas = pd.read_excel('datos.xlsx', sheet_name=None)
```

JSON:

```
# Cargar JSON
df = pd.read_json('datos.json')

# JSON anidado (requiere normalizaci n)
import json
with open('datos.json', 'r') as f:
    data = json.load(f)
df = pd.json_normalize(data)
```

1.2.2 Conexión a bases de datos

```
import sqlite3
import pandas as pd

# Conectar a SQLite
conn = sqlite3.connect('base_datos.db')

# Ejecutar query y cargar en DataFrame
query = "SELECT * FROM ventas WHERE fecha >= '2024-01-01'"
df = pd.read_sql_query(query, conn)

# Cerrar conexi n
conn.close()
```

Para PostgreSQL, MySQL, etc., se usa `sqlalchemy`:

```
from sqlalchemy import create_engine

# Crear conexi n
engine = create_engine('postgresql://user:pass@localhost/db')

# Cargar datos
df = pd.read_sql_query("SELECT * FROM tabla", engine)
```

1.2.3 Acceso a APIs

```
import requests
import pandas as pd

# Hacer request a API
url = "https://api.ejemplo.com/datos"
headers = {'Authorization': 'Bearer TU_TOKEN'}
response = requests.get(url, headers=headers)

# Verificar que fue exitoso
if response.status_code == 200:
    data = response.json()
    df = pd.DataFrame(data)
else:
    print(f"Error: {response.status_code}")
```

1.3 Carga de datos en R

1.3.1 Tidyverse para archivos

El ecosistema `tidyverse` provee herramientas eficientes para carga de datos.

CSV:

```
library(readr)

# Cargar CSV
df <- read_csv('datos.csv')

# Con opciones
df <- read_csv(
  'datos.csv',
```

```
  col_types = cols(
    edad = col_integer(),
    fecha = col_date(format = "%Y-%m-%d")
  ),
  na = c("NA", "", "N/A")
)
```

Excel:

```
library(readxl)

# Cargar Excel
df <- read_excel('datos.xlsx', sheet = 'Hoja1')

# Ver nombres de hojas
excel_sheets('datos.xlsx')
```

JSON:

```
library(jsonlite)

# Cargar JSON
data <- fromJSON('datos.json')

# Si es una lista, convertir a data frame
df <- as.data.frame(data)
```

1.3.2 Conexión a bases de datos

```
library(DBI)
library(RSQLite)

# Conectar a SQLite
conn <- dbConnect(RSQLite::SQLite(), "base_datos.db")

# Ejecutar query
df <- dbGetQuery(conn, "SELECT * FROM ventas")

# Cerrar conexi n
dbDisconnect(conn)
```

Para PostgreSQL:

```
library(RPostgres)

conn <- dbConnect(
  RPostgres::Postgres(),
  dbname = 'mi_base',
  host = 'localhost',
  user = 'usuario',
  password = 'clave'
)

df <- dbGetQuery(conn, "SELECT * FROM tabla")
dbDisconnect(conn)
```

1.3.3 Acceso a APIs

```r
library(httr)
library(jsonlite)

# Hacer request
url <- "https://api.ejemplo.com/datos"
response <- GET(url, add_headers(Authorization = "Bearer TOKEN"))

# Verificar status
if (status_code(response) == 200) {
  data <- content(response, as = "text")
  df <- fromJSON(data)
} else {
  print(paste("Error:", status_code(response)))
}
```

1.4 Exploración inicial

Una vez cargados los datos, siempre realizar una exploración inicial:

Python:

```python
# Dimensiones
print(df.shape)  # (filas, columnas)

# Primeras filas
df.head()

# Informaci n general (tipos, memoria)
df.info()

# Estad sticas descriptivas
df.describe()

# Nombres de columnas
df.columns

# Tipos de datos
df.dtypes
```

R:

```r
# Dimensiones
dim(df)  # c(filas, columnas)

# Primeras filas
head(df)

# Estructura
str(df)

# Resumen estad stico
summary(df)

# Nombres de columnas
names(df)

# Tipos de datos
```

```
sapply(df, class)
```

2 Clase 4: Calidad de Datos

2.1 Dimensiones de la calidad de datos

La calidad de los datos se evalúa en varias dimensiones:

1. **Completitud:** ¿Hay valores faltantes?
2. **Consistencia:** ¿Los datos son coherentes entre sí?
3. **Precisión:** ¿Los datos son correctos?
4. **Actualidad:** ¿Los datos están actualizados?
5. **Validez:** ¿Los datos cumplen con las reglas de negocio?

2.2 Valores faltantes

Los valores faltantes (missing values) son uno de los problemas más comunes en datasets reales.

2.2.1 Tipos de valores faltantes

- **MCAR (Missing Completely At Random):** La ausencia no tiene relación con ninguna variable
- **MAR (Missing At Random):** La ausencia está relacionada con variables observadas
- **MNAR (Missing Not At Random):** La ausencia está relacionada con el valor faltante mismo

Ejemplos de tipos de missingness

MCAR: Un sensor falla aleatoriamente
MAR: Personas jóvenes no responden pregunta sobre ingresos (relacionado con edad, no con ingresos)
MNAR: Personas con ingresos altos no responden pregunta sobre ingresos (relacionado con el valor mismo)

2.2.2 Detección de valores faltantes

Python:

```
# Contar valores faltantes por columna
df.isnull().sum()

# Porcentaje de valores faltantes
(df.isnull().sum() / len(df)) * 100

# Visualizar patr n de missingness
import missingno as msno
msno.matrix(df)
msno.heatmap(df)  # Correlaci n de missingness
```

R:

```r
# Contar valores faltantes
colSums(is.na(df))

# Porcentaje
colMeans(is.na(df)) * 100

# Visualizar con naniar
library(naniar)
vis_miss(df)
gg_miss_var(df)
```

2.2.3 Estrategias para tratar valores faltantes

1. Eliminación

- **Listwise deletion:** Eliminar filas con algún valor faltante
- **Pairwise deletion:** Usar solo datos disponibles para cada análisis
- **Por columna:** Eliminar columnas con demasiados faltantes

Cuidado con la eliminación

Eliminar puede causar:

- Pérdida de información valiosa
- Sesgos si los datos no son MCAR
- Reducción significativa del tamaño del dataset

Regla general: Solo eliminar si <5% de faltantes y son MCAR

Python - Eliminación:

```python
# Eliminar filas con cualquier NA
df_clean = df.dropna()

# Eliminar filas donde columnas espec ficas tienen NA
df_clean = df.dropna(subset=['columna1', 'columna2'])

# Eliminar columnas con >50% de NAs
umbral = len(df) * 0.5
df_clean = df.dropna(thresh=umbral, axis=1)
```

R - Eliminación:

```r
# Eliminar filas con cualquier NA
df_clean <- na.omit(df)

# Eliminar filas donde columnas espec ficas tienen NA
df_clean <- df %>%
  filter(!is.na(columna1), !is.na(columna2))

# Eliminar columnas con >50% NAs
umbral <- nrow(df) * 0.5
df_clean <- df %>%
  select(where(~ sum(is.na(.)) < umbral))
```

2. Imputación simple
Reemplazar valores faltantes con un valor estadístico:

- **Media/Mediana:** Para variables numéricas
- **Moda:** Para variables categóricas
- **Valor constante:** Como 0, "desconocido", etc.

Python - Imputación simple:

```
from sklearn.impute import SimpleImputer

# Imputar con media
imputer = SimpleImputer(strategy='mean')
df[['columna1', 'columna2']] = imputer.fit_transform(
    df[['columna1', 'columna2']]
)

# Imputar con moda (m s frecuente)
imputer = SimpleImputer(strategy='most_frequent')
df['categoria'] = imputer.fit_transform(df[['categoria']])

# Imputar con constante
df['columna'].fillna(0, inplace=True)
```

R - Imputación simple:

```
library(tidyverse)

# Imputar con media
df <- df %>%
  mutate(columna1 = ifelse(is.na(columna1),
                           mean(columna1, na.rm = TRUE),
                           columna1))

# Imputar con mediana
df <- df %>%
  mutate(across(where(is.numeric),
                ~replace_na(., median(., na.rm = TRUE))))

# Imputar categor as con moda
moda <- function(x) {
  ux <- unique(x[!is.na(x)])
  ux[which.max(tabulate(match(x, ux)))]
}

df <- df %>%
  mutate(categoria = replace_na(categoria, moda(categoria)))
```

3. Imputación avanzada
Métodos más sofisticados que consideran relaciones entre variables:

- **KNN (K-Nearest Neighbors):** Imputa usando valores de observaciones similares
- **Regresión:** Predice valores faltantes usando otras variables
- **MICE (Multiple Imputation by Chained Equations):** Imputación múltiple iterativa
- **Interpolación:** Para series temporales

Python - KNN:

```
from sklearn.impute import KNNImputer

# Imputar con KNN (k=5 vecinos)
imputer = KNNImputer(n_neighbors=5)
df_imputed = pd.DataFrame(
    imputer.fit_transform(df),
    columns=df.columns
)
```

R - MICE:

```
library(mice)

# Imputaci n m ltiple
imputed <- mice(df, m = 5, method = 'pmm', seed = 42)

# Obtener dataset completo
df_complete <- complete(imputed, 1)  # Primera imputaci n
```

2.3 Duplicados

2.3.1 Detección de duplicados

Python:

```
# Detectar duplicados (todas las columnas)
duplicados = df.duplicated()
print(f"Duplicados totales: {duplicados.sum()}")

# Detectar duplicados por columnas espec ficas
duplicados = df.duplicated(subset=['id', 'fecha'])

# Ver las filas duplicadas
df[duplicados]
```

R:

```
# Detectar duplicados
duplicados <- duplicated(df)
cat("Duplicados totales:", sum(duplicados), "\n")

# Por columnas espec ficas
duplicados <- df %>%
  duplicated(by = c('id', 'fecha'))

# Ver duplicados
df[duplicados, ]
```

2.3.2 Eliminación de duplicados

Python:

```
# Eliminar duplicados (mantiene primera ocurrencia)
df_clean = df.drop_duplicates()

# Por columnas espec ficas
df_clean = df.drop_duplicates(subset=['id', 'fecha'])
```

```
# Mantener  ltima  ocurrencia
df_clean = df.drop_duplicates(keep='last')
```

R:

```
# Eliminar duplicados
df_clean <- distinct(df)

# Por columnas espec ficas
df_clean <- df %>%
  distinct(id, fecha, .keep_all = TRUE)
```

2.4 Inconsistencias

Las inconsistencias son errores en los datos que violan reglas lógicas o de negocio.

2.4.1 Tipos de inconsistencias

1. **Valores fuera de rango:** Edad = -5, temperatura = 500°C
2. **Formatos incorrectos:** Fechas como "32/13/2024"
3. **Inconsistencias entre variables:** Fecha de nacimiento posterior a fecha de graduación
4. **Errores tipográficos:** "Bahia Blanca", "B. Blanca", "Bahía blanca" (mismo valor, diferentes strings)
5. **Valores imposibles:** Porcentajes >100%, probabilidades <0

2.4.2 Detección de inconsistencias

Python:

```
# Valores fuera de rango
print("Edades negativas:", (df['edad'] < 0).sum())
print("Edades >120:", (df['edad'] > 120).sum())

# Verificar rangos esperados
print(f"Edad min: {df['edad'].min()}, max: {df['edad'].max()}")

# Valores  nicos  (buscar variaciones)
print(df['ciudad'].value_counts())

# Inconsistencias entre columnas
inconsistentes = df[df['fecha_nacimiento'] > df['fecha_registro']]
print(f"Registros inconsistentes: {len(inconsistentes)}")
```

R:

```
# Valores fuera de rango
cat("Edades negativas:", sum(df$edad < 0, na.rm = TRUE), "\n")
cat("Edades >120:", sum(df$edad > 120, na.rm = TRUE), "\n")

# Rangos
cat("Edad min:", min(df$edad, na.rm = TRUE),
    "max:", max(df$edad, na.rm = TRUE), "\n")

# Valores  nicos
```

```r
table(df$ciudad)

# Inconsistencias
inconsistentes <- df %>%
  filter(fecha_nacimiento > fecha_registro)
cat("Registros inconsistentes:", nrow(inconsistentes), "\n")
```

2.4.3 Corrección de inconsistencias

Estandarización de texto:

```python
# Python - Estandarizar texto
df['ciudad'] = df['ciudad'].str.lower().str.strip()

# Reemplazar variaciones
df['ciudad'] = df['ciudad'].replace({
    'b. blanca': 'bahia blanca',
    'bahia blanca': 'bah a blanca'
})
```

```r
# R - Estandarizar texto
df <- df %>%
  mutate(ciudad = str_to_lower(str_trim(ciudad)))

# Reemplazar variaciones
df <- df %>%
  mutate(ciudad = recode(ciudad,
    'b. blanca' = 'bahia blanca',
    'bahia blanca' = 'bah a blanca'
  ))
```

Corrección de valores fuera de rango:

```python
# Python - Reemplazar valores imposibles con NA
df.loc[df['edad'] < 0, 'edad'] = np.nan
df.loc[df['edad'] > 120, 'edad'] = np.nan

# O clip (limitar valores)
df['edad'] = df['edad'].clip(lower=0, upper=120)
```

```r
# R - Reemplazar con NA
df <- df %>%
  mutate(edad = ifelse(edad < 0 | edad > 120, NA, edad))

# O con pmin/pmax
df <- df %>%
  mutate(edad = pmin(pmax(edad, 0), 120))
```

3 Clase 5: Transformación de Datos

3.1 Conversión de tipos de datos

Los datos pueden cargarse con tipos incorrectos. Es fundamental corregirlos.

3.1.1 Tipos de datos comunes

- **Numéricos:** int, float
- **Texto:** string, categorical
- **Booleanos:** True/False
- **Fechas:** datetime, date
- **Categóricos ordenados:** ordinal

Python:

```
# Convertir a num rico
df['edad'] = pd.to_numeric(df['edad'], errors='coerce')

# Convertir a string
df['id'] = df['id'].astype(str)

# Convertir a fecha
df['fecha'] = pd.to_datetime(df['fecha'], format='%Y-%m-%d')

# Convertir a categ rico
df['categoria'] = df['categoria'].astype('category')

# Categ rico ordenado
df['educacion'] = pd.Categorical(
    df['educacion'],
    categories=['primaria', 'secundaria', 'universitaria'],
    ordered=True
)
```

R:

```
# Convertir a num rico
df$edad <- as.numeric(df$edad)

# Convertir a string
df$id <- as.character(df$id)

# Convertir a fecha
library(lubridate)
df$fecha <- ymd(df$fecha)

# Convertir a factor
df$categoria <- as.factor(df$categoria)

# Factor ordenado
df$educacion <- factor(
  df$educacion,
  levels = c('primaria', 'secundaria', 'universitaria'),
  ordered = TRUE
)
```

3.2 Creación de variables derivadas

Crear nuevas variables a partir de las existentes (feature engineering básico).

Python:

```python
# Operaciones aritm ticas
df['imc'] = df['peso'] / (df['altura'] ** 2)

# Operaciones con fechas
df['edad'] = (pd.Timestamp.now() - df['fecha_nacimiento']).dt.days /
    365.25

# Extraer componentes de fecha
df['a  o'] = df['fecha'].dt.year
df['mes'] = df['fecha'].dt.month
df['dia_semana'] = df['fecha'].dt.dayofweek

# Operaciones con texto
df['nombre_completo'] = df['nombre'] + ' ' + df['apellido']
df['longitud_nombre'] = df['nombre'].str.len()

# Condiciones
df['adulto'] = df['edad'] >= 18
df['categoria_edad'] = pd.cut(
    df['edad'],
    bins=[0, 18, 65, 100],
    labels=['joven', 'adulto', 'mayor']
)
```

R:

```r
library(tidyverse)

df <- df %>%
  mutate(
    # Operaciones aritm ticas
    imc = peso / (altura^2),

    # Operaciones con fechas
    edad = as.numeric(difftime(Sys.Date(), fecha_nacimiento, units = "
        days")) / 365.25,
    a  o = year(fecha),
    mes = month(fecha),
    dia_semana = wday(fecha),

    # Operaciones con texto
    nombre_completo = paste(nombre, apellido),
    longitud_nombre = nchar(nombre),

    # Condiciones
    adulto = edad >= 18,
    categoria_edad = cut(edad,
                         breaks = c(0, 18, 65, 100),
                         labels = c('joven', 'adulto', 'mayor'))
  )
```

3.3 Codificación de variables categóricas

Los algoritmos de machine learning requieren datos numéricos. Las variables categóricas deben codificarse.

3.3.1 Label Encoding

Asigna un número único a cada categoría.

Cuidado con Label Encoding

Solo usar label encoding cuando las categorías tienen un orden natural (ordinal).
Para categorías nominales, usar one-hot encoding para evitar que el modelo asuma relaciones ordinales falsas.

Python:

```
from sklearn.preprocessing import LabelEncoder

# Label encoding
le = LabelEncoder()
df['educacion_encoded'] = le.fit_transform(df['educacion'])

# Ver mapeo
print(dict(zip(le.classes_, le.transform(le.classes_))))
```

R:

```
# Label encoding (autom tico con factor)
df$educacion_encoded <- as.numeric(factor(df$educacion))

# Con orden espec fico
df$educacion_encoded <- as.numeric(factor(
  df$educacion,
  levels = c('primaria', 'secundaria', 'universitaria')
))
```

3.3.2 One-Hot Encoding

Crea una columna binaria para cada categoría.

Ejemplo:

Original	color_rojo	color_verde	color_azul
rojo	1	0	0
verde	0	1	0
azul	0	0	1
rojo	1	0	0

Python:

```
# One-hot encoding
df_encoded = pd.get_dummies(df, columns=['color'], prefix='color')

# Con sklearn (m s control)
from sklearn.preprocessing import OneHotEncoder

encoder = OneHotEncoder(sparse=False, drop='first')  # drop primera para
    evitar multicolinealidad
encoded = encoder.fit_transform(df[['color']])

# Convertir a DataFrame
df_encoded = pd.DataFrame(
    encoded,
```

```
    columns=encoder.get_feature_names_out(['color'])
)
```

R:

```
# One-hot encoding
library(caret)

dummies <- dummyVars(" ~ color", data = df)
df_encoded <- predict(dummies, newdata = df)

# Con tidyverse
df_encoded <- df %>%
  mutate(value = 1) %>%
  pivot_wider(names_from = color,
              values_from = value,
              values_fill = 0,
              names_prefix = "color_")
```

4 Clase 6: Normalización e Integración

4.1 Normalización y Estandarización

Muchos algoritmos de ML son sensibles a la escala de las variables. La normalización/estandarización ayuda a:

- Mejorar convergencia en algoritmos de optimización
- Dar igual peso a todas las variables
- Mejorar interpretabilidad de coeficientes

4.1.1 Min-Max Scaling (Normalización)

Escala valores al rango [0, 1]:

$$X_{norm} = \frac{X - X_{min}}{X_{max} - X_{min}}$$

Python:

```
from sklearn.preprocessing import MinMaxScaler

scaler = MinMaxScaler()
df[['edad', 'ingreso']] = scaler.fit_transform(df[['edad', 'ingreso']])

# Con rango personalizado [-1, 1]
scaler = MinMaxScaler(feature_range=(-1, 1))
```

R:

```
# Normalizaci n manual
normalizar <- function(x) {
  (x - min(x, na.rm = TRUE)) / (max(x, na.rm = TRUE) - min(x, na.rm =
     TRUE))
}

df <- df %>%
  mutate(across(c(edad, ingreso), normalizar))
```

```r
# Con caret
preproc <- preProcess(df[, c('edad', 'ingreso')], method = "range")
df[, c('edad', 'ingreso')] <- predict(preproc, df[, c('edad', 'ingreso')
    ])
```

4.1.2 Standardization (Z-score)

Transforma a media 0 y desviación estándar 1:

$$X_{std} = \frac{X - \mu}{\sigma}$$

Python:

```python
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
df[['edad', 'ingreso']] = scaler.fit_transform(df[['edad', 'ingreso']])
```

R:

```r
# Estandarizaci n
df <- df %>%
  mutate(across(c(edad, ingreso), scale))

# Con caret
preproc <- preProcess(df[, c('edad', 'ingreso')], method = c("center", "
    scale"))
df[, c('edad', 'ingreso')] <- predict(preproc, df[, c('edad', 'ingreso')
    ])
```

¿Cuándo usar cada uno?

Min-Max:

- Cuando necesitas valores en un rango específico
- Neural networks, KNN
- No hay outliers extremos

Standardization:

- Cuando asumes distribución normal
- Regresión logística, SVM, PCA
- Presencia de outliers (más robusto)

4.2 Discretización (Binning)

Convertir variables continuas en categóricas dividiendo en intervalos.

Python:

```python
# Bins de igual ancho
df['edad_grupo'] = pd.cut(
    df['edad'],
    bins=5,  # 5 intervalos de igual tama o
```

```
    labels=['muy_joven', 'joven', 'adulto', 'mayor', 'muy_mayor']
)

# Bins con puntos de corte espec ficos
df['edad_grupo'] = pd.cut(
    df['edad'],
    bins=[0, 18, 30, 50, 65, 100],
    labels=['menor', 'joven', 'adulto', 'maduro', 'mayor']
)

# Bins de igual frecuencia (quantiles)
df['ingreso_grupo'] = pd.qcut(
    df['ingreso'],
    q=4,  # 4 cuartiles
    labels=['bajo', 'medio_bajo', 'medio_alto', 'alto']
)
```

R:

```
# Bins de igual ancho
df <- df %>%
  mutate(edad_grupo = cut(
    edad,
    breaks = 5,
    labels = c('muy_joven', 'joven', 'adulto', 'mayor', 'muy_mayor')
  ))

# Bins con puntos espec ficos
df <- df %>%
  mutate(edad_grupo = cut(
    edad,
    breaks = c(0, 18, 30, 50, 65, 100),
    labels = c('menor', 'joven', 'adulto', 'maduro', 'mayor')
  ))

# Bins de igual frecuencia
df <- df %>%
  mutate(ingreso_grupo = cut(
    ingreso,
    breaks = quantile(ingreso, probs = seq(0, 1, 0.25), na.rm = TRUE),
    labels = c('bajo', 'medio_bajo', 'medio_alto', 'alto'),
    include.lowest = TRUE
  ))
```

4.3 Integración de datos

A menudo necesitamos combinar datos de múltiples fuentes.

4.3.1 Joins (uniones)

Similar a SQL, existen diferentes tipos de joins:

- **Inner join:** Solo filas con match en ambas tablas
- **Left join:** Todas las filas de la izquierda + matches de la derecha
- **Right join:** Todas las filas de la derecha + matches de la izquierda

- **Outer join:** Todas las filas de ambas tablas

Python:

```
# Inner join
df_merged = pd.merge(df1, df2, on='id', how='inner')

# Left join
df_merged = pd.merge(df1, df2, on='id', how='left')

# Join con nombres diferentes de columnas
df_merged = pd.merge(
    df1, df2,
    left_on='cliente_id',
    right_on='id',
    how='left'
)

# Join con m ltiples columnas
df_merged = pd.merge(df1, df2, on=['id', 'fecha'], how='inner')
```

R:

```
library(dplyr)

# Inner join
df_merged <- inner_join(df1, df2, by = "id")

# Left join
df_merged <- left_join(df1, df2, by = "id")

# Join con nombres diferentes
df_merged <- left_join(
  df1, df2,
  by = c("cliente_id" = "id")
)

# Join con m ltiples columnas
df_merged <- inner_join(df1, df2, by = c("id", "fecha"))
```

4.3.2 Concatenación

Apilar DataFrames verticalmente u horizontalmente.

Python:

```
# Concatenar verticalmente (apilar filas)
df_combined = pd.concat([df1, df2], axis=0, ignore_index=True)

# Concatenar horizontalmente (a adir columnas)
df_combined = pd.concat([df1, df2], axis=1)
```

R:

```
# Concatenar verticalmente
df_combined <- bind_rows(df1, df2)

# Concatenar horizontalmente
df_combined <- bind_cols(df1, df2)
```

5 Clase 7: APIs, Web Scraping y Tidy Data

5.1 APIs REST

Las APIs REST permiten obtener datos de servicios web de forma estructurada.

5.1.1 Componentes de una API REST

- **Endpoint:** URL del recurso
- **Métodos HTTP:** GET (obtener), POST (crear), PUT (actualizar), DELETE (eliminar)
- **Headers:** Metadatos (autenticación, tipo de contenido)
- **Parámetros:** Query parameters o body
- **Respuesta:** Generalmente JSON

Python - Ejemplo completo:

```
import requests
import pandas as pd

# API p blica de ejemplo: JSONPlaceholder
url = "https://jsonplaceholder.typicode.com/posts"

# GET request simple
response = requests.get(url)

if response.status_code == 200:
    data = response.json()
    df = pd.DataFrame(data)
    print(df.head())
else:
    print(f"Error: {response.status_code}")

# GET con par metros
params = {'userId': 1}
response = requests.get(url, params=params)

# GET con autenticaci n
headers = {'Authorization': 'Bearer TU_TOKEN'}
response = requests.get(url, headers=headers)

# POST request
nuevo_post = {
    'title': 'Nuevo post',
    'body': 'Contenido del post',
    'userId': 1
}
response = requests.post(url, json=nuevo_post)
```

R - Ejemplo completo:

```
library(httr)
library(jsonlite)

# API p blica
url <- "https://jsonplaceholder.typicode.com/posts"
```

```r
# GET request
response <- GET(url)

if (status_code(response) == 200) {
  data <- content(response, as = "text")
  df <- fromJSON(data)
  head(df)
} else {
  print(paste("Error:", status_code(response)))
}

# GET con par metros
response <- GET(url, query = list(userId = 1))

# GET con autenticaci n
response <- GET(
  url,
  add_headers(Authorization = "Bearer TU_TOKEN")
)

# POST request
nuevo_post <- list(
  title = 'Nuevo post',
  body = 'Contenido',
  userId = 1
)
response <- POST(url, body = nuevo_post, encode = "json")
```

5.2 Web Scraping

Cuando no hay API disponible, se puede extraer datos directamente del HTML.

5.2.1 Herramientas

- **Python:** BeautifulSoup, Scrapy, Selenium (para páginas dinámicas)
- **R:** rvest, xml2

5.2.2 Proceso de web scraping

1. Inspeccionar la estructura HTML de la página
2. Identificar selectores CSS o XPath
3. Descargar el HTML
4. Extraer la información
5. Limpiar y estructurar los datos

Python - BeautifulSoup:

```python
import requests
from bs4 import BeautifulSoup
import pandas as pd

# Descargar p gina
url = "https://ejemplo.com/tabla"
```

```python
response = requests.get(url)
soup = BeautifulSoup(response.content, 'html.parser')

# Encontrar todos los enlaces
enlaces = soup.find_all('a')
for link in enlaces:
    print(link.get('href'))

# Extraer tabla
tabla = soup.find('table', {'class': 'datos'})
filas = tabla.find_all('tr')

data = []
for fila in filas[1:]:  # Saltar encabezado
    celdas = fila.find_all('td')
    data.append([celda.text.strip() for celda in celdas])

df = pd.DataFrame(data, columns=['col1', 'col2', 'col3'])

# Extraer por clase CSS
elementos = soup.find_all('div', {'class': 'precio'})
precios = [elem.text for elem in elementos]

# Extraer por selector CSS
elementos = soup.select('.producto .precio')
```

R - rvest:

```r
library(rvest)
library(tidyverse)

# Descargar p gina
url <- "https://ejemplo.com/tabla"
pagina <- read_html(url)

# Extraer todos los enlaces
enlaces <- pagina %>%
  html_nodes("a") %>%
  html_attr("href")

# Extraer tabla
tabla <- pagina %>%
  html_node("table.datos") %>%
  html_table()

df <- as_tibble(tabla)

# Extraer por clase CSS
precios <- pagina %>%
  html_nodes(".precio") %>%
  html_text()

# Extraer por selector CSS
elementos <- pagina %>%
  html_nodes(".producto .precio") %>%
  html_text()
```

Mejores prácticas de web scraping

1. Revisar robots.txt: `https://sitio.com/robots.txt`
2. Respetar términos de servicio
3. Usar rate limiting (pausas entre requests)
4. Identificarse con User-Agent apropiado
5. No sobrecargar el servidor
6. Cachear resultados

Python - Rate limiting:

```
import time

urls = ['url1', 'url2', 'url3']
for url in urls:
    response = requests.get(url)
    # Procesar respuesta
    time.sleep(1)  # Esperar 1 segundo entre requests
```

5.3 Tidy Data

Los **datos ordenados** (tidy data) siguen tres principios:

1. Cada variable forma una columna
2. Cada observación forma una fila
3. Cada tipo de unidad observacional forma una tabla

Ejemplo: Datos no ordenados vs ordenados

No ordenado (wide):

Persona	Año_2020	Año_2021
Ana	50000	52000
Luis	48000	49000

Ordenado (long):

Persona	Año	Ingreso
Ana	2020	50000
Ana	2021	52000
Luis	2020	48000
Luis	2021	49000

5.4 Reshape: Pivot y Melt

5.4.1 Pivot (wide → long)

Convertir columnas en filas.

Python:

```python
# Melt (wide      long)
df_long = pd.melt(
    df,
    id_vars=['persona'],            # Columnas a mantener
    value_vars=['a o_2020', 'a o_2021'],  # Columnas a convertir
    var_name='a  o',                 # Nombre de la nueva columna de
        variables
    value_name='ingreso'            # Nombre de la nueva columna de
        valores
)

# Simplificado (todas las columnas excepto id_vars)
df_long = pd.melt(df, id_vars=['persona'], var_name='a  o', value_name='
    ingreso')
```

R:

```r
# Pivot longer (wide      long)
df_long <- df %>%
  pivot_longer(
    cols = c(a  o_2020, a  o_2021),  # Columnas a convertir
    names_to = "a  o",               # Nombre de columna de nombres
    values_to = "ingreso"            # Nombre de columna de valores
  )

# Con prefijo a remover
df_long <- df %>%
  pivot_longer(
    cols = starts_with("a  o_"),
    names_to = "a  o",
    names_prefix = "a  o_",
    values_to = "ingreso"
  )
```

5.4.2 Pivot (long → wide)

Convertir filas en columnas.

Python:

```python
# Pivot (long      wide)
df_wide = df_long.pivot(
    index='persona',       # Columna que define las filas
    columns='a  o',         # Columna que se convierte en columnas
    values='ingreso'       # Valores a rellenar
)

# Pivot con agregaci n (si hay duplicados)
df_wide = df_long.pivot_table(
    index='persona',
    columns='a  o',
    values='ingreso',
    aggfunc='mean'  # Funci n de agregaci n
)
```

R:

```r
# Pivot wider (long      wide)
df_wide <- df_long %>%
```

```r
  pivot_wider(
    names_from = a_o,       # Columna que se convierte en columnas
    values_from = ingreso   # Valores a rellenar
  )

# Con prefijo
df_wide <- df_long %>%
  pivot_wider(
    names_from = a_o,
    values_from = ingreso,
    names_prefix = "a_o_"
  )
```

Resumen de la Unidad 1

- Los datos provienen de **múltiples fuentes**: archivos, bases de datos, APIs, web scraping
- **Pandas (Python)** y **tidyverse (R)** son las herramientas principales para carga y manipulación
- La **calidad de datos** se evalúa en: completitud, consistencia, precisión, actualidad, validez
- Los **valores faltantes** requieren estrategias apropiadas: eliminación o imputación
- Siempre verificar **duplicados** e **inconsistencias**
- La **transformación** incluye: conversión de tipos, creación de variables, codificación de categóricas
- La **normalización/estandarización** es crucial para muchos algoritmos de ML
- La **integración** de datos se hace mediante joins y concatenación
- Las **APIs REST** proveen acceso estructurado a datos
- El **web scraping** permite extraer datos de páginas web
- Los **datos ordenados** (tidy data) facilitan el análisis

Conceptos clave

- Fuentes de datos: CSV, Excel, JSON, SQL, APIs
- Calidad de datos
- Valores faltantes: MCAR, MAR, MNAR
- Imputación: simple, KNN, MICE
- Duplicados e inconsistencias
- Conversión de tipos
- Feature engineering
- Label encoding vs one-hot encoding
- Normalización vs estandarización
- Discretización (binning)

- Joins: inner, left, right, outer
- REST API
- Web scraping: BeautifulSoup, rvest
- Tidy data
- Pivot, melt, reshape

Próxima unidad

En la Unidad 2 nos enfocaremos en la **Exploración y Visualización de Datos**: estadística descriptiva, análisis de distribuciones, detección de outliers, y creación de visualizaciones efectivas con matplotlib, seaborn y ggplot2.